

VEXTA NETWORK

TECHNICAL HANDOVER & ADMINISTRATION GUIDE

DOCUMENT TYPE

Technical Handover Guide

DELIVERY DATE

July 2026

TOTAL SERVICES

**Prisma / MongoDB / Plisio /
Resend**

DEPLOYMENT STATUS

✓ Production Active

[Complete System Audit Resources](#)

Vexta Network — Technical Handover & System Administration Guide

This document contains the system architecture details, database configuration guides, server architectures, and operational instructions for the Vexta Network platform. It is designed to assist another developer in auditing and taking over project management.

1. Project Overview

Vexta is an AI Arbitrage & Trading Platform built with a modern TypeScript/React stack.

- **Core Framework:** Next.js 16 (React 19, TailwindCSS, TypeScript)
 - **Database Layer:** Prisma ORM (v6.19.0) with MongoDB adapter
 - **Email Service:** Resend (Transactional emails & OTPs)
 - **Crypto Payment Gateway:** Plisio (USDT BEP-20 on Binance Smart Chain)
 - **Production Domain:** <https://vexta.network>
-

2. Database Setup & Configuration

The platform uses **Prisma ORM** connecting to a **MongoDB Atlas Cluster**.

Connection String Template

Configure the `DATABASE_URL` environment variable inside your local `.env` file using the following structure:

```
DATABASE_URL="mongodb+srv://[username]:[password]@[host]/vexta?retryWrites=true"
```

- **Adapter / Schema:** `prisma/schema.prisma` defines the collections:
 - `User` : Main accounts, balance tracking, downline references, MLM parameters.
 - `Plan` : Starter, Advance, and Ultra investment tiers.
 - `Investment` : Active contracts, capital, elapsed days, and ROI logs.
 - `DailyROIEntry` : Audit rows for daily profit distributions.
 - `PlisioInvoice` : Invoice transactions generated for user deposits.
 - `BatchPayoutRun` : Log of Friday batch payout CSV generations.
 - `ReferralLink` / `Commission` : MLM structure and level payouts.
 - `Transaction` / `Withdrawal` : Financial logs and withdrawal request states.
 - `Settings` : Global system toggles (Maintenance mode, gateway control, promo rules).
-

3. Server & SSL Infrastructure

- **Hosting Provider:** The frontend and API routes are currently hosted on **Vercel** (`vercel.json` configurations). The stack can also be deployed to a standard Ubuntu VPS with Node.js 18+ (requires standard configurations for PM2 or Systemd).
 - **SSL Certificates:** Automated SSL certificates are provisioned dynamically by the hosting platform (Vercel / Cloudflare). No manual certificate installations are required.
 - **DNS Resolution:** Managed via the domain registrar pointing to Vercel's nameservers (`ns1.vercel-dns.com` / `ns2.vercel-dns.com`).
-

4. Third-Party Integrations

1. Resend (Email Integration)

- **API Key:** `[Securely provided by owner]`
- **Domain Verification:** The sender domain (e.g. `vexta.network`) must be verified via SPF/DKIM records inside the Resend dashboard.
- **Features:** Sends account verification codes (OTP), withdrawal confirmations, support updates, and daily cron run status report logs.

2. Plisio (USDT BEP-20 Payment Gateway)

- **Secret Key:** [Securely provided by owner]
 - **Auto-Forwarding:** Configure your corporate destination wallet in the Plisio dashboard setting. Plisio automatically routes incoming customer payments to this address.
 - **Webhook URL:** `https://[your-domain]/api/plisio/webhook`
 - **Robust Verification Mechanism:**
 - The webhook route `app/api/plisio/webhook/route.ts` supports standard IPN signature validations.
 - To maximize safety and prevent bugs from varying payload structures, the system performs a **direct backend API fallback call** to `https://api.plisio.net/api/v1/invoices/${txn_id}?api_key=${secretKey}` to query Plisio's servers directly for the final, official transaction status.
-

5. Core Business Logic & Algorithms

A developer taking over the platform must understand the following three systems:

1. Daily ROI Distribution & Compounding

- **Route:** `/api/admin/run-daily-roi` (triggered Mon–Fri).
- **Authorization:** Gated by JWT admin session OR by passing the `CRON_SECRET` via the `x-cron-key` header or `?secret=` query parameter.
- **Logic** (`server/services/earnings.service.ts`):
 - Computes `1.0%` daily interest.
 - Incorporates compounding with a **2-day pending delay**: today's distributed returns are added to `pendingCapital` and only merge into the compounding base (`activeCapital`) after 2 business days.
 - Every investment has a **hard ceiling** of `200%` payout (principal × 2). Once the sum of all earnings (ROI + MLM commissions) hits this cap, the contract status transitions to `completed` and profit generation stops.

2. Multi-Level Marketing (MLM) Unilevel Commission Propagation

- **Trigger:** Automatically executes upon successful activation of a user's investment package.

- **Logic** ([lib/referral-engine.ts](#)):
 - Distributes commissions up the sponsor chain for **13 tiers** of uplines:
 - Level 1: 8% | Level 2: 5% | Level 3: 3% | Level 4: 3% | Level 5: 2% | Level 6: 2% | Level 7: 1% | Levels 8–13: 0.5%.
 - Enforces the **200% Hard Cap**: When a commission propagates to a sponsor, the capacity of the sponsor's active packages is assessed. Commissions that fit within the capacity are added to their balance; any excess overflows and is **permanently forfeited**.
 - Logs separate audit events for gross commissions generated and actual net credited commissions.

3. Administrative Batch Payout Processing

- **Route**: [app/api/admin/batch-payout/route.ts](#)
- **Workflow**:
 1. Admin requests a batch generation (`action: "generate"`).
 2. The system scans all pending withdrawal requests and calculates the Net Payout after processing fees (6% for withdrawals under \$600, 2% for \$600+).
 3. Formats and exports a CSV file matching Plisio's mass-payment protocol: `address,amount,currency` .
 4. Executing the payout (`action: "execute"`) is gated by a 2FA OTP sent to the admin's email. On execution, withdrawal statuses are marked as `approved` and ledger entries are logged.

6. Administrative & Database Utility Scripts

The `scripts/` directory contains standard helper scripts for DB management and audit operations:

- [scripts/check-plisio-api.ts](#): Tests Plisio endpoint health and returns invoice states.
- [scripts/check-user-deposit.ts](#): Manually checks the blockchain status of a transaction ID.
- [scripts/clean-db.ts](#): Resets all user balances to 0 (useful for environment test resets).
- [scripts/clean-db-delete.ts](#): Drops test entries.

- [scripts/credit-mismatch-deposit.ts](#): Corrects deposits that generated amount discrepancies on Plisio.
- [scripts/reset-data.ts](#): Completely purges users and investments to reset the database.
- [scripts/seed-plans.ts](#): Upserts default STARTER, ADVANCE, and ULTRA contracts.

To execute any script (using ts-node):

```
npx ts-node --project tsconfig.json scripts/check-plisio-api.ts
```

7. Local Development & Deployment Guide

For a new developer to set up the codebase locally:

Prerequisite Installation

Ensure Node.js 18+ and npm/pnpm/yarn are installed on the local system.

```
# 1. Install dependencies
npm install

# 2. Configure Local Environment Variables
# Copy the env template and fill in the secrets (provided securely by the owner)
cp .env.example .env

# 3. Generate Prisma Client
# Builds the Prisma client types locally based on schema.prisma
npx prisma generate

# 4. Seed Database
# Seeds default investment tiers and configures the default admin account:
# Admin Email: admin@vexta.app | Password: Admin@1234!
npx prisma db seed

# 5. Launch Local Dev Server
# Starts the application on http://localhost:3000
npm run dev
```

Production Deployment

To build the static next package and run in production:

```
# Production Build
npm run build

# Start Production Server
npm start
```

Note: Ensure all Environment Variables (.env keys) are configured in your hosting platform dashboard.